이 파트에서는 생성형 AI를 프로그래밍으로 활용하기 위한 **API 사용법**을 학습합니다.

OpenAI(GPT), **Google(Gemini)**, **Anthropic(Claude)** 세 가지 플랫폼의 API Key 발급부터 SDK 설치, Chat Completions API, 최신 Responses API까지 범용적으로 다룹니다.

> 📌 ***참고:*** *이 파트부터 Python 코드로 API를 직접 호출합니다. GPT를 메인 예제로 사용하되, Gemini와 Claude의 코드도 함께 제공합니다.*

학습 목표

- GPT, Gemini, Claude API Key를 발급하고 안전하게 관리할 수 있습니다.
- Chat Completions API의 메시지 구조와 주요 파라미터를 이해합니다.
- 동일 프롬프트로 세 플랫폼의 응답을 비교하는 프로그램을 만들 수 있습니다.
- Responses API와 멀티모달 입력 처리 방법을 이해합니다.

이 파트의 구성

챕터	제목	핵심 내용
Chapter 13	API 기초와 개발환경 추가 설정	API Key 발급, 토큰, 요금, SDK 설치
Chapter 14	Chat Completions API 활용	메시지 구조, 파라미터, 스트리밍, 멀티턴
Chapter 15	Responses API와 최신 동향	마이그레이션, 멀티모달, 최신 SDK

이번 챕터에서는 GPT, Gemini, Claude 세 가지 AI 플랫폼의 API를 직접 코드로 호출하기 위한 기초 지식과 개발환경 설정을 다룹니다. API가 무엇인지, 어떻게 키를 발급받고 안전하게 관리하는지, 그리고 각 플랫폼의 비용 구조가 어떻게 다른지를 단계별로 살펴봅니다.

이 챕터를 마치면 다음을 할 수 있습니다.

- API와 REST API의 기본 개념을 설명할 수 있습니다.
- OpenAI, Google Gemini, Anthropic Claude 각각의 API 키를 발급받고 환경변수로 등록할 수 있습니다.
- 세 플랫폼의 요금 체계와 토큰 비용 계산 방식을 이해합니다.
- `.env` 파일로 API 키를 안전하게 관리할 수 있습니다.
- 각 플랫폼 API를 파이썬 코드로 처음 호출하는 실습을 완료합니다.

> ⚠️ ***Part 5를 시작하기 전에 — 모델 ID는 자주 바뀝니다***
>
> *이 책의 본문과 실습 코드에는 집필 시점(2026년 상반기)의 대표 모델 ID — 예를 들어 `gpt-4.1-mini`, `gemini-2.5-flash`, `claude-haiku-4-5` 같은 이름 — 가 등장합니다. 하지만 OpenAI · Google · Anthropic 모두* ***수개월 단위로 새 모델을 출시하고, 기존 모델을 점진적으로 폐기(deprecate)*** *합니다.*
>
> *따라서 실습 도중 다음과 같은 오류를 만날 수 있습니다.*
>
> ```
> openai.NotFoundError: The model `gpt-4.1-mini` does not exist or you do not have access to it.
> google.api_core.exceptions.NotFound: 404 models/gemini-2.5-flash is not found ...
> anthropic.NotFoundError: model: claude-haiku-4-5
> ```
>
> *이 경우* ***책의 코드가 잘못된 것이 아니라 모델 ID가 갱신된 것****이므로, 아래 공식 페이지에서 동급(저가형/표준형/추론형)의 최신 모델 ID로 한 줄만 바꿔 사용하면 됩니다.*
>
> - *OpenAI 모델 목록: `https://platform.openai.com/docs/models`*
> - *Google Gemini 모델 목록: `https://ai.google.dev/gemini-api/docs/models`*
> - *Anthropic Claude 모델 목록: `https://docs.anthropic.com/en/docs/about-claude/models`*
>
> *또한* ***요금·컨텍스트 길이·무료 티어 한도****도 함께 바뀌므로 본문 표는 참고 수준으로 보고, 결제 직전에는 항상 각 플랫폼의 공식 Pricing 페이지에서 다시 확인합니다. Part 5~6의 모든 코드는 모델 ID만 교체하면 동일한 흐름으로 동작하도록 설계되어 있습니다.*

13.1 API란 무엇인가 — REST API의 기본 개념

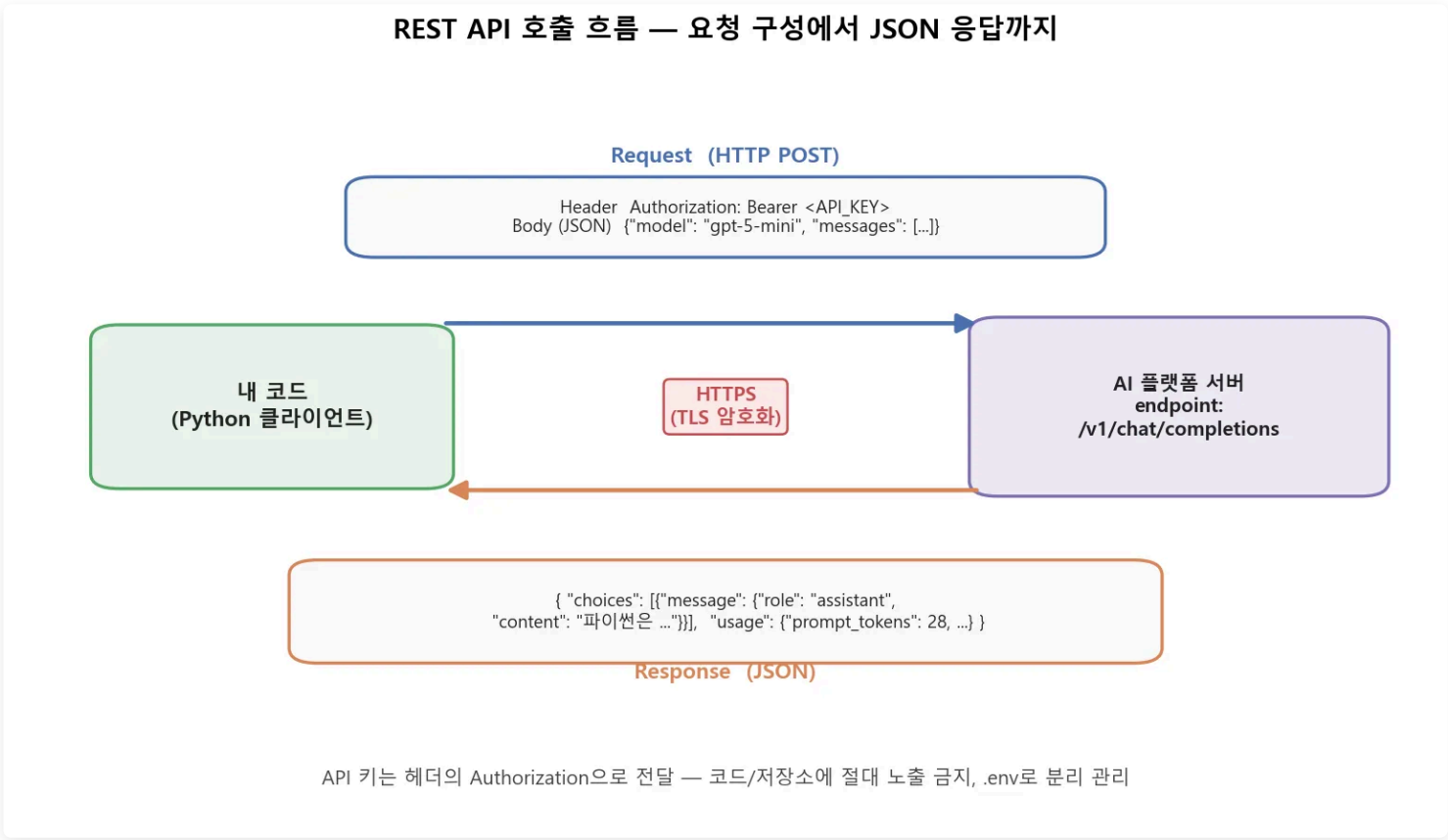
API(Application Programming Interface)는 서로 다른 소프트웨어가 정해진 규칙으로 대화하는 창구입니다. 식당으로 비유하면, 손님(내 코드)이 메뉴를 보고 종업원(API)에게 주문하면 주방(AI 서버)이 요리를 만들어 가져다주는 구조입니다.

REST API(Representational State Transfer API)는 웹에서 가장 널리 쓰이는 API 설계 방식입니다. HTTP 프로토콜을 사용해 요청을 보내고 응답을 받으며, 데이터는 주로 **JSON(JavaScript Object Notation)** 형식으로 주고받습니다.

REST API의 핵심 요소는 다음과 같습니다.

요소	설명	예시
엔드포인트(Endpoint)	API 서버의 주소	`https://api.openai.com/v1/chat/completions`
HTTP 메서드	요청의 종류	GET(조회), POST(생성), PUT(수정), DELETE(삭제)
헤더(Header)	요청 부가 정보	인증 키, 콘텐츠 타입 등
바디(Body)	요청 본문 데이터	JSON 형식의 메시지, 모델명 등
응답(Response)	서버가 돌려주는 결과	JSON 형식의 AI 답변

AI API를 호출할 때는 항상 `POST` 메서드를 사용하고, 헤더에 **API 키(API Key)**를 담아 자신이 정당한 사용자임을 증명합니다.



내 코드(Python 클라이언트)가 헤더에 API 키, 바디에 JSON 메시지를 담아 **HTTPS POST** 로 AI 서버 엔드포인트를 호출하면, 서버는 모델 응답을 다시 JSON으로 돌려줍니다. 모든 통신은 TLS로 암호화되며 API 키는 헤더의 `Authorization` 필드를 통해서만 전달됩니다.

> 💡 ***API 키는 신분증입니다.*** *키가 노출되면 다른 사람이 내 이름으로 비용을 발생시킬 수 있으니 절대 공개 저장소에 올리지 않도록 주의합니다.*

13.2 OpenAI API Key 발급 및 환경변수 등록

OpenAI API 키는 GPT 모델을 코드로 호출하기 위한 인증 수단입니다.

발급 절차

1단계 — OpenAI 계정 생성 및 로그인

`https://platform.openai.com` 에 접속합니다. 구글 계정, 마이크로소프트 계정, 또는 이메일로 회원가입합니다. 이미 ChatGPT 계정이 있다면 같은 계정으로 로그인합니다.

2단계 — API Keys 페이지 이동

로그인 후 화면 왼쪽 사이드바에서 `API Keys` 메뉴를 클릭합니다. 또는 `https://platform.openai.com/api-keys` 로 직접 접속합니다.

3단계 — 새 키 생성

`+ Create new secret key` 버튼을 클릭합니다. 키 이름(Name)을 입력합니다(예: `my-api-key`). `Create secret key` 버튼을 클릭합니다.

4단계 — 키 복사 및 저장

생성된 키는 `sk-proj-...` 또는 `sk-...` 형태입니다. 이 화면을 벗어나면 키를 다시 볼 수 없으니 반드시 복사해 안전한 곳에 저장합니다.

> ⚠️ ***주의:*** *키를 잃어버렸다면 새로 발급받아야 합니다. 분실이나 노출이 의심될 경우 즉시 `Revoke` 버튼으로 폐기합니다.*

5단계 — 결제 수단 등록

API 사용에는 비용이 발생합니다. `Settings > Billing` 에서 신용카드를 등록합니다. 소액(예: $5~$10)을 충전하여 시작합니다.

환경변수 등록

발급받은 키를 운영체제 환경변수에 등록합니다.

Windows (PowerShell)

```
[System.Environment]::SetEnvironmentVariable("OPENAI_API_KEY", "sk-...", "User")
```

macOS / Linux (bash 또는 zsh)

```
echo 'export OPENAI_API_KEY="sk-..."' >> ~/.bashrc
source ~/.bashrc
```

등록 후 터미널을 새로 열어 다음 명령으로 확인합니다.

```
echo $OPENAI_API_KEY
```

13.3 Google Gemini API Key 발급 및 등록

Google Gemini API는 Google AI Studio를 통해 키를 발급받습니다.

발급 절차

1단계 — Google AI Studio 접속

`https://aistudio.google.com` 에 접속합니다. 구글 계정으로 로그인합니다.

2단계 — API 키 생성

화면 왼쪽 사이드바 또는 상단 메뉴에서 `Get API key` 버튼을 클릭합니다. `Create API key` 를 선택합니다. 기존 Google Cloud 프로젝트를 선택하거나 새 프로젝트를 생성합니다. 생성된 키는 `AIza...` 형태입니다.

3단계 — 키 복사

`Copy` 버튼으로 키를 클립보드에 복사하여 안전한 곳에 저장합니다.

> 💡 ***무료 티어 활용**: Gemini API는 분당 요청 수가 제한되는 무료 티어를 제공합니다. 학습 목적으로 처음 시작할 때는 무료 티어로 충분합니다.*

환경변수 등록

Windows (PowerShell)

```
[System.Environment]::SetEnvironmentVariable("GOOGLE_API_KEY", "AIza ... ", "User")
```

macOS / Linux

```
echo 'export GOOGLE_API_KEY="AIza ... "' >> ~/.bashrc
source ~/.bashrc
```

13.4 Anthropic Claude API Key 발급 및 등록

Anthropic Claude API는 Anthropic Console을 통해 발급받습니다.

발급 절차

1단계 — Anthropic Console 접속

`https://console.anthropic.com` 에 접속합니다. 구글 계정 또는 이메일로 회원가입 후 로그인합니다.

2단계 — API Keys 페이지 이동

로그인 후 화면 상단 메뉴 또는 사이드바에서 `API Keys` 를 클릭합니다. 또는
`https://console.anthropic.com/settings/keys` 로 직접 접속합니다.

3단계 — 새 키 생성

`+ Create Key` 버튼을 클릭합니다. 키 이름을 입력합니다(예: `my-api-key`). `Create Key` 버튼을 클릭합니다.

4단계 — 키 복사

생성된 키는 `sk-ant- ...` 형태입니다. 반드시 이 화면에서 복사해두어야 합니다. 이후 화면을 벗어나면 다시 확인할 수 없습니다.

5단계 — 결제 수단 등록

`Settings > Billing` 에서 결제 수단을 등록하고 크레딧을 충전합니다.

환경변수 등록

Windows (PowerShell)

```
[System.Environment]::SetEnvironmentVariable("ANTHROPIC_API_KEY", "sk-ant-...", "User")
```

macOS / Linux

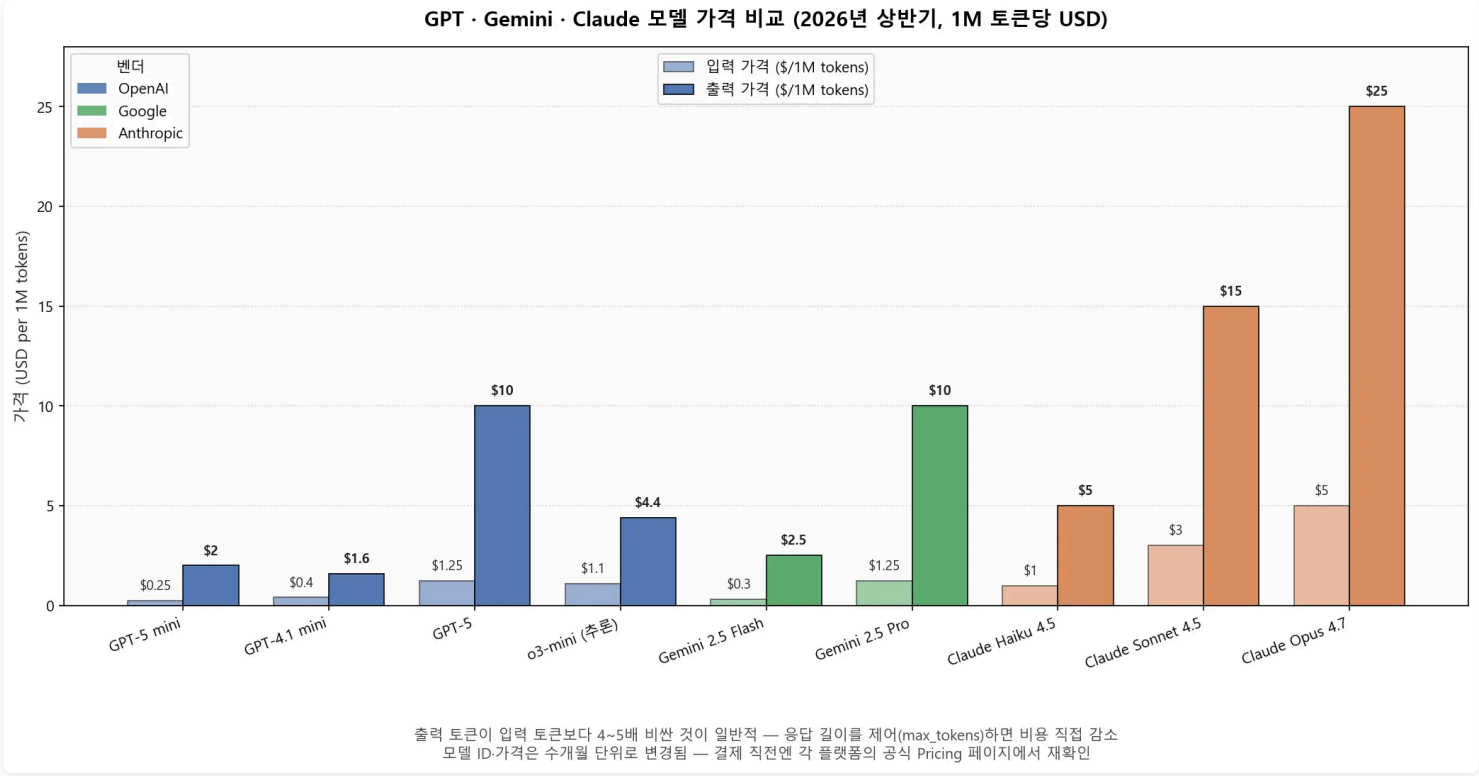
```
echo 'export ANTHROPIC_API_KEY="sk-ant-..."' >> ~/.bashrc
source ~/.bashrc
```

13.5 API 요금 체계 비교 (GPT / Gemini / Claude)

세 플랫폼 모두 **토큰(Token)** 단위로 과금합니다. 입력 토큰과 출력 토큰을 따로 계산하며, 일반적으로 출력 토큰이 더 비쌉니다.

> 📌 ***아래 요금은 2026년 상반기 기준 대표 모델의 예시입니다.*** *요금은 수시로 변경되므로 실제 사용 전 각 플랫폼 공식 요금 페이지를 반드시 확인합니다.*

플랫폼	대표 모델	입력 (1M 토큰)	출력 (1M 토큰)	컨텍스트	무료 티어
OpenAI	GPT-5	$1.25 | $10.00	400K	없음 (크레딧 소진 후 유료)	
OpenAI	GPT-5 mini	$0.25 | $2.00	400K	일부 무료 티어	
OpenAI	GPT-4.1 mini	$0.40 | $1.60	1M	없음	
OpenAI	o3-mini (추론)	$1.10 | $4.40	200K	없음	
Google	Gemini 2.5 Flash	$0.30 | $2.50	1M	있음 (분당 RPM 제한)	
Google	Gemini 2.5 Pro	$1.25 (>200K 입력 시 $2.50)	$10.00 (>200K $15.00)	1M	있음 (낮은 RPM)
Anthropic	Claude Haiku 4.5	$1.00 | $5.00	200K	없음	
Anthropic	Claude Sonnet 4.5	$3.00 | $15.00	200K (1M 옵션)	일부 무료 (claude.ai)	
Anthropic	Claude Opus 4.5 / 4.7	$5.00 | $25.00	1M	없음	



벤더별 색상(OpenAI 파랑 · Google 초록 · Anthropic 주황)으로 9개 모델의 1M 토큰당 입력/출력 가격을 한눈에 비교할 수 있습니다. **출력 가격이 입력보다 4~5배 비싼 것**이 일반적이라 `max_tokens` 같은 출력 길이 제어가 곧 비용 절감으로 직결됩니다.

> 💡 ***학습 시작 팁**: 처음에는 가장 저렴한 모델(GPT-5 mini · GPT-4.1 mini, Gemini 2.5 Flash, Claude Haiku 4.5)로 실습하고, 품질 차이가 필요할 때 상위 모델로 한 줄만 바꿔 호출합니다.*
>
> *단, 위 모델 ID와 가격은 **언제든 바뀔 수 있습니다.** 챕터 도입부의 모델 ID는 자주 바뀝니다 안내를 참고해, 오류가 나면 동급의 최신 모델로 교체하면 됩니다.*

각 플랫폼의 공식 요금 페이지는 다음과 같습니다.

- OpenAI: `https://openai.com/pricing`
- Google: `https://ai.google.dev/pricing`
- Anthropic: `https://www.anthropic.com/pricing`

13.6 토큰(Token)의 개념과 비용 계산

토큰이란

토큰(Token)은 AI 모델이 텍스트를 처리하는 최소 단위입니다. 단어와 비슷하지만 정확히 같지는 않습니다.

- 영문 기준: 1 토큰 ≈ 0.75 단어 (약 4글자)

- 한글 기준: 1 글자 ≈ 1~3 토큰 (한글은 영문보다 토큰 소비가 많습니다)

예를 들어 `"Hello, world!"` 는 약 4 토큰이고, `"안녕하세요, 세상아!"` 는 약 8~12 토큰입니다.

토큰 계산 예시

파이썬에서 `tiktoken` 라이브러리로 OpenAI 모델의 토큰 수를 직접 계산할 수 있습니다.

```
import tiktoken

# GPT-4o 모델의 인코더 불러오기
enc = tiktoken.encoding_for_model("gpt-4o")

text = "안녕하세요. 오늘 날씨가 정말 좋네요!"
tokens = enc.encode(text)

print(f"텍스트: {text}")
print(f"토큰 수: {len(tokens)}")
print(f"토큰 목록: {tokens}")
```

비용 계산 공식

```
비용 = (입력 토큰 수 / 1,000,000) × 입력 단가
     + (출력 토큰 수 / 1,000,000) × 출력 단가
```

예를 들어 GPT-5 mini(입력 $0.25 / 출력 $2.00)로 입력 1,000 토큰, 출력 500 토큰을 사용했을 때의 비용입니다.

```
입력 비용 = (1,000 / 1,000,000) × $0.25 = $0.00025
출력 비용 = (500  / 1,000,000) × $2.00 = $0.00100
합계      = $0.00125 (약 1.7원, 2026년 환율 기준)
```

> 💡 ***비용 관리 팁**: 각 플랫폼 콘솔의 `Usage` 또는 `Billing` 페이지에서 누적 사용량과 비용을 실시간으로 확인할 수 있습니다. 월 한도(Spending Limit)를 설정하면 예상치 못한 과금을 방지할 수 있습니다.*

13.7 필수 라이브러리 설치 — openai, google-genai, anthropic

세 플랫폼 API를 파이썬에서 편리하게 호출하기 위해 공식 라이브러리를 설치합니다.

각 라이브러리 소개

라이브러리	제공사	설치 패키지명	비고
openai	OpenAI	`openai`	1.x 이상. Chat Completions와 Responses API 모두 지원
google-genai	Google	`google-genai`	2025년 이후 공식 권장 SDK
anthropic	Anthropic	`anthropic`	0.40 이상 권장

> 💡 ***Gemini SDK 주의**: 과거에 널리 쓰이던 `google-generativeai` 패키지(`import google.generativeai as genai`)는 점차 **신규 SDK `google-genai` 로 마이그레이션**되고 있습니다. 새로 시작하는 프로젝트는 `google-genai` 를 권장하며, 기존 코드는 한동안 그대로 동작하지만 신기능(최신 Gemini 모델 일부)은 신 SDK에서만 제공될 수 있습니다.*

설치 명령

터미널(또는 명령 프롬프트)에서 다음 명령을 실행합니다.

```
pip install openai google-genai anthropic
```

`.env` 파일 관리를 위한 라이브러리도 함께 설치합니다.

```
pip install python-dotenv
```

한 번에 모두 설치할 수 있습니다.

```
pip install openai google-genai anthropic python-dotenv
```

설치 확인

파이썬 인터프리터에서 임포트가 정상적으로 되는지 확인합니다.

```
import openai
import anthropic
from google import genai  # google-genai 신 SDK

print("openai 버전:", openai.__version__)
print("anthropic 버전:", anthropic.__version__)
print("google-genai 임포트 성공")
print("모든 라이브러리 정상 설치 완료!")
```

오류 없이 버전 번호가 출력되면 설치가 완료된 것입니다.

> ⚠️ ***가상환경 사용 권장**: 프로젝트별로 `python -m venv venv` 명령으로 가상환경을 만들고, 그 안에 라이브러리를 설치하면 다른 프로젝트와 버전 충돌을 방지할 수 있습니다.*

13.8 환경변수와 .env 파일로 Key 안전하게 관리하기

API 키는 절대 코드에 직접 입력해서는 안 됩니다. 코드를 깃허브 등에 공개할 경우 키가 노출되어 타인이 내 계정으로 비용을 발생시킬 수 있습니다.

.env 파일 방식

.env 파일은 환경변수를 파일로 관리하는 표준적인 방법입니다. 프로젝트 루트 폴더에 `.env` 파일을 만들고 키를 저장합니다.

`.env` 파일 예시입니다.

```
OPENAI_API_KEY=sk-proj-xxxxxxxxxxxxxxxxxxxx
GOOGLE_API_KEY=AIzaxxxxxxxxxxxxxxxxxxxxxxxxx
ANTHROPIC_API_KEY=sk-ant-xxxxxxxxxxxxxxxxxxx
```

파이썬 코드에서 `python-dotenv` 라이브러리로 이 파일을 불러옵니다.

```
from dotenv import load_dotenv
import os

# .env 파일의 환경변수를 불러옵니다
load_dotenv()

openai_key = os.getenv("OPENAI_API_KEY")
google_key = os.getenv("GOOGLE_API_KEY")
anthropic_key = os.getenv("ANTHROPIC_API_KEY")

print("OpenAI 키 앞 10자:", openai_key[:10] if openai_key else "미설정")
print("Google 키 앞 10자:", google_key[:10] if google_key else "미설정")
print("Anthropic 키 앞 10자:", anthropic_key[:10] if anthropic_key else "미설정")
```

.gitignore 설정

`.env` 파일은 반드시 `.gitignore`에 추가하여 깃 저장소에 올라가지 않도록 합니다. 프로젝트 폴더에 `.gitignore` 파일을 만들고 다음 내용을 추가합니다.

```
# API 키 파일 제외
.env
.env.local
.env.*.local
```

보안 체크리스트

항목	권장 여부
코드에 키 직접 입력	절대 금지
`.env` 파일에 저장	권장
`.env`를 `.gitignore`에 추가	필수
깃허브 등 공개 저장소에 키 포함	절대 금지
키 정기적 교체	권장
플랫폼 콘솔에서 월 한도 설정	권장

> 📌 ***혹시 실수로 키를 커밋했다면**: 즉시 해당 키를 플랫폼 콘솔에서 폐기(Revoke)하고 새 키를 발급받아야 합니다. 깃 히스토리에서 삭제해도 이미 노출된 키는 계속 유효하므로 폐기가 우선입니다.*

[실습] GPT / Gemini / Claude API 각각 첫 호출 테스트

지금까지 설정한 환경을 바탕으로 세 플랫폼 API를 각각 호출해봅니다. 모든 코드는 같은 질문("파이썬이란 무엇인지 한 문장으로 설명해줘")을 각 플랫폼에 보내고 답변을 출력합니다.

실습 준비

프로젝트 폴더 구조를 다음과 같이 준비합니다.

```
my_ai_project/
├── .env          ← API 키 저장
├── .gitignore    ← .env 제외 설정
└── ch13_test.py  ← 실습 코드
```

실습 1 — OpenAI GPT API 첫 호출

```
# ch13_gpt_test.py
from dotenv import load_dotenv
import os
from openai import OpenAI

# 환경변수 로드
load_dotenv()

# 클라이언트 초기화 (API 키는 환경변수에서 자동으로 읽습니다)
client = OpenAI()

# API 호출
response = client.chat.completions.create(
    model="gpt-5-mini",            # 2026년 기준 저비용 일반 모델. 폐기 시 gpt-4.1-mini 등으로 교체
    messages=[
        {
            "role": "user",
            "content": "파이썬이란 무엇인지 한 문장으로 설명해줘"
        }
    ],
    max_tokens=100                 # 최대 응답 토큰 수 제한
)

# 응답 텍스트 추출
answer = response.choices[0].message.content
print("GPT 응답:", answer)

# 토큰 사용량 확인
usage = response.usage
print(f"입력 토큰: {usage.prompt_tokens}, 출력 토큰: {usage.completion_tokens}")
```

예상 출력

```
GPT 응답: 파이썬은 간결하고 읽기 쉬운 문법을 가진 범용 프로그래밍 언어로, 데이터 분석, 웹 개발, 인공지능 등 다양한 분야에서 널리 사용됩니다.
입력 토큰: 28, 출력 토큰: 52
```

실습 2 — Google Gemini API 첫 호출

신규 `google-genai` SDK 사용 예시입니다(2025년 이후 공식 권장).

```
# ch13_gemini_test.py
from dotenv import load_dotenv
import os
from google import genai

# 환경변수 로드
load_dotenv()

# 클라이언트 초기화 (GOOGLE_API_KEY 환경변수를 자동으로 사용)
client = genai.Client()

# API 호출
response = client.models.generate_content(
    model="gemini-2.5-flash",      # 2026년 기준 무료 티어 가능 빠른 모델. 폐기 시 최신 Flash 계열로 교체
    contents="파이썬이란 무엇인지 한 문장으로 설명해줘",
)

# 응답 텍스트 추출
print("Gemini 응답:", response.text)

# 토큰 사용량 확인
usage = response.usage_metadata
print(f"입력 토큰: {usage.prompt_token_count}, 출력 토큰: {usage.candidates_token_count}")
```

> 💡 *기존 `google-generativeai` (구 SDK)를 그대로 쓰는 경우, `genai.configure( ... )` → `genai.GenerativeModel("gemini-2.5-flash").generate_content( ... )` 형태로 호출합니다. 새 코드는 위의 `google-genai` 신 SDK 패턴을 권장합니다.*

예상 출력

```
Gemini 응답: 파이썬은 가독성 높은 문법과 방대한 라이브러리 생태계를 갖춘 인터프리터 방식의 범용 프로그래밍 언어입니다.
입력 토큰: 22, 출력 토큰: 38
```

실습 3 — Anthropic Claude API 첫 호출

```
# ch13_claude_test.py
from dotenv import load_dotenv
import os
import anthropic

# 환경변수 로드
load_dotenv()

# 클라이언트 초기화 (API 키는 환경변수에서 자동으로 읽습니다)
client = anthropic.Anthropic()

# API 호출
response = client.messages.create(
    model="claude-haiku-4-5",          # 2026년 기준 저비용 Haiku 모델. 폐기 시 최신 Haiku 계열로 교체
    max_tokens=100,
    messages=[
        {
            "role": "user",
            "content": "파이썬이란 무엇인지 한 문장으로 설명해줘"
        }
    ]
)

# 응답 텍스트 추출
answer = response.content[0].text
print("Claude 응답:", answer)

# 토큰 사용량 확인
usage = response.usage
print(f"입력 토큰: {usage.input_tokens}, 출력 토큰: {usage.output_tokens}")
```

예상 출력

```
Claude 응답: 파이썬은 문법이 간결하고 배우기 쉬우며, 웹 개발, 데이터 과학, 인공지능 등 다양한 분야에서 활용되는 고수준 프로그래밍 언어입니다.
입력 토큰: 30, 출력 토큰: 55
```

세 API 비교 요약

항목	OpenAI GPT	Google Gemini	Anthropic Claude
클라이언트 생성	`OpenAI()`	`genai.Client()`	`anthropic.Anthropic()`
호출 메서드	`chat.completions.create()`	`client.models.generate_content()`	`messages.create()`
응답 텍스트 경로	`response.choices[0].message.content`	`response.text`	`response.content[0].text`
모델 파라미터명	`model`	`model`	`model`
환경변수명	`OPENAI_API_KEY`	`GOOGLE_API_KEY`	`ANTHROPIC_API_KEY`

> 💡 ***실습 체크**: 세 코드를 모두 실행했을 때 오류 없이 응답이 출력된다면 이번 챕터의 목표를 달성한 것입니다. 오류가 발생하면 API 키 등록 여부와 라이브러리 설치 상태를 다시 확인합니다.*

마치며

이번 챕터에서는 GPT, Gemini, Claude 세 AI 플랫폼의 API를 사용하기 위한 기초 환경을 모두 갖추었습니다.

핵심 내용 요약

- **API**는 소프트웨어 간의 통신 창구이며, **REST API**는 HTTP 기반의 표준 방식입니다.
- OpenAI, Google AI Studio, Anthropic Console에서 각각 **API 키**를 발급받을 수 있습니다.
- **토큰**은 AI 모델의 과금 단위이며, 입력과 출력 토큰이 별도로 계산됩니다.
- `openai`, `google-genai`, `anthropic` 라이브러리로 각 플랫폼을 파이썬에서 쉽게 호출할 수 있습니다.
- API 키는 `.env` 파일에 저장하고 `.gitignore`로 보호하는 것이 필수입니다.

다음 챕터 예고

Chapter 14 Chat Completions API에서는 이번 챕터에서 구성한 환경을 바탕으로 OpenAI의 Chat Completions API를 깊이 있게 다룹니다. 시스템 메시지, 대화 히스토리 관리, 스트리밍 응답 등 실전에서 자주 쓰이는 고급 기능을 학습합니다.

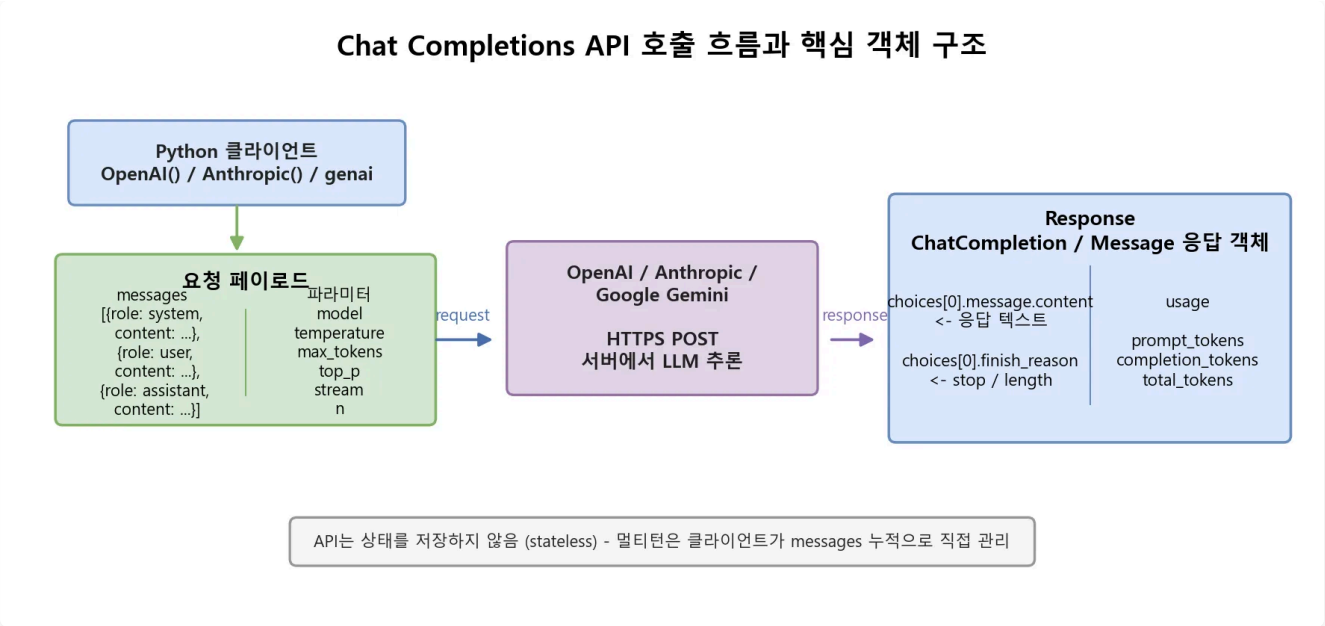
Chapter 14. Chat Completions API 활용

이번 챕터에서는 **Chat Completions API**를 중심으로 OpenAI, Google Gemini, Anthropic Claude 세 가지 주요 AI 서비스의 API 호출 방법을 배웁니다. 각 API의 구조와 파라미터를 이해하고, 실제로 실행 가능한 Python 코드를 통해 멀티턴 대화 구현과 스트리밍 응답까지 단계적으로 익혀 봅니다.

> ***이 챕터를 마치면**: OpenAI Chat Completions API의 구조를 이해하고, temperature 등 핵심 파라미터를 조절할 수 있습니다. 스트리밍 응답을 구현하고, 멀티턴 대화를 위한 컨텍스트 관리 방법을 익힙니다. 나아가 Gemini와 Claude API도 동일한 방식으로 호출할 수 있게 됩니다.*
>
> 📌 ***[2026 업데이트] 최신 모델 매핑***
>
> *이 챕터의 코드 예시는 학습 편의를 위해 2024년 출시 모델 ID(`gpt-4o-mini` , `gemini-1.5-flash` , `claude-3-5-haiku-20241022`)를 사용합니다. **2026년 5월 기준 동급 최신 모델**로 바꿔 실행하려면 다음 매핑을 사용하세요.*
>
챕터 코드 ID	*2026년 동급 최신 모델*
`gpt-4o-mini`	`gpt-5-mini` (또는 `gpt-5`)
`gemini-1.5-flash`	`gemini-2.5-flash`
`gemini-1.5-pro`	`gemini-2.5-pro`
`claude-3-5-haiku-20241022`	`claude-haiku-4-5`
>
> *Gemini SDK도 신 `google-genai` 패키지로 마이그레이션이 진행 중입니다. 자세한 안내는 Ch13의 모델 ID·SDK 갱신 박스를 참고하세요.*

14.1 OpenAI Chat Completions API 구조

Chat Completions API는 OpenAI가 제공하는 핵심 API로, 대화 형식의 메시지 목록을 입력받아 모델의 응답을 반환합니다. `gpt-4o` , `gpt-4o-mini` 등의 모델을 사용할 수 있습니다.



기본 호출 구조

```
from openai import OpenAI

client = OpenAI(api_key="YOUR_API_KEY")  # 환경변수 OPENAI_API_KEY 설정 시 생략 가능

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "당신은 친절한 AI 어시스턴트입니다."},
        {"role": "user", "content": "파이썬이란 무엇인가요?"}
    ]
)

# 응답 텍스트 추출
print(response.choices[0].message.content)
```

응답 객체 구조

API가 반환하는 응답 객체(`ChatCompletion`)의 주요 필드는 다음과 같습니다.

필드	설명
`id`	응답 고유 ID
`object`	객체 타입 (`chat.completion`)
`model`	사용된 모델명
`choices`	생성된 응답 목록 (기본 1개)
`choices[0].message.content`	실제 응답 텍스트
`choices[0].finish_reason`	종료 이유 (`stop` , `length` 등)
`usage.prompt_tokens`	입력 토큰 수
`usage.completion_tokens`	출력 토큰 수
`usage.total_tokens`	전체 토큰 수

```
# 응답 객체 상세 확인 예시
print(f"모델: {response.model}")
print(f"응답: {response.choices[0].message.content}")
print(f"종료 이유: {response.choices[0].finish_reason}")
print(f"사용 토큰: {response.usage.total_tokens}")
```

14.2 메시지 역할 (system / user / assistant)

Chat Completions API는 대화를 메시지 목록으로 관리합니다. 각 메시지는 `role` 과 `content` 두 가지 필드로 구성됩니다. 역할 (`role`)은 세 가지가 있습니다.

역할	설명	사용 시점
`system`	모델의 행동 방식과 페르소나 설정	대화 맨 처음에 한 번
`user`	사용자의 입력 메시지	매 질문마다
`assistant`	모델의 이전 응답	대화 이력 유지 시

```
from openai import OpenAI

client = OpenAI()

messages = [
    # system: 모델의 역할과 지침 설정
    {
        "role": "system",
        "content": (
            "당신은 파이썬 전문 튜터입니다. "
            "초보자도 이해할 수 있도록 쉽고 간결하게 설명하세요. "
            "코드 예시는 반드시 포함하세요."
        )
    },
    # user: 사용자 질문
    {
        "role": "user",
        "content": "리스트 컴프리헨션이 뭔가요?"
    },
    # assistant: 이전 응답 (멀티턴 대화 시 이력으로 포함)
    {
        "role": "assistant",
        "content": "리스트 컴프리헨션은 기존 리스트를 기반으로 새 리스트를 간결하게 만드는 문법입니다."
    },
    # 후속 질문
    {
        "role": "user",
        "content": "실제 예제를 보여주세요."
    }
]

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=messages
)

print(response.choices[0].message.content)
```

> ***system 메시지 활용 팁**: system 메시지는 모델의 성격, 응답 언어, 출력 형식, 금지 사항 등을 명시하는 데 활용합니다. 구체적일수록 원하는 방향의 응답을 얻기 쉽습니다.*

14.3 주요 파라미터 상세

Chat Completions API에는 응답의 품질과 다양성을 조절하는 다양한 파라미터가 있습니다.

temperature

temperature는 응답의 무작위성(창의성)을 조절합니다. 값의 범위는 `0.0` ~ `2.0` 이며, 낮을수록 일관되고 예측 가능한 응답, 높을수록 다양하고 창의적인 응답이 생성됩니다.

값 범위	특성	적합한 용도
`0.0` ~ `0.3`	일관적, 결정론적	코드 생성, 사실 질의, 번역
`0.4` ~ `0.7`	균형적	일반 대화, 요약
`0.8` ~ `1.5`	창의적, 다양	창작, 아이디어 브레인스토밍

max_tokens

max_tokens는 응답으로 생성할 최대 토큰 수를 제한합니다. 비용 절감과 응답 길이 제어에 활용합니다.

> *1 토큰 ≈ 영어 4글자 또는 한글 1~2글자. 응답이 잘리면 `finish_reason` 이 `length` 로 표시됩니다.*

top_p

top_p (nucleus sampling)는 누적 확률 상위 p% 내의 토큰만 고려하여 샘플링합니다. `temperature` 와 유사하게 다양성을 조절하지만, 일반적으로 둘 중 하나만 조절하는 것을 권장합니다.

frequency_penalty

frequency_penalty는 이미 등장한 단어의 반복을 억제합니다. 값 범위는 `-2.0` ~ `2.0` 이며, 양수 값은 반복을 줄이고 음수 값은 반복을 허용합니다.

presence_penalty

presence_penalty는 아직 등장하지 않은 새로운 주제나 단어를 유도합니다. `frequency_penalty` 가 빈도 기반 패널티라면, `presence_penalty` 는 등장 여부 기반 패널티입니다.

파라미터 종합 예시

```
from openai import OpenAI

client = OpenAI()

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "당신은 창의적인 이야기 작가입니다."},
        {"role": "user", "content": "외계인이 서울에 나타난 이야기를 짧게 써주세요."}
    ],
    temperature=1.2,          # 창의적인 응답을 위해 높게 설정
    max_tokens=300,            # 최대 300 토큰
    top_p=0.95,                # 상위 95% 토큰 풀에서 샘플링
    frequency_penalty=0.5,    # 단어 반복 억제
    presence_penalty=0.3,     # 새로운 주제 유도
    n=1                        # 응답 후보 개수 (기본값 1)
)

print(response.choices[0].message.content)
```

14.4 응답 파싱과 에러 처리

실제 서비스에서는 API 호출 실패에 대비한 안전한 에러 처리가 필수입니다.

응답 파싱

```
from openai import OpenAI

client = OpenAI()

def get_completion(prompt: str, system: str = "당신은 유용한 AI 어시스턴트입니다.") → str:
    """프롬프트를 받아 응답 텍스트를 반환하는 헬퍼 함수"""
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": system},
            {"role": "user", "content": prompt}
        ],
        max_tokens=500
    )

    # choices가 비어 있는 경우 방어 처리
    if not response.choices:
        return ""

    message = response.choices[0].message
    finish_reason = response.choices[0].finish_reason

    # 응답이 잘린 경우 경고
    if finish_reason == "length":
        print("⚠️ 경고: max_tokens 한도로 응답이 잘렸습니다.")

    return message.content or ""

result = get_completion("파이썬의 장점을 세 가지로 요약해주세요.")
print(result)
```

에러 처리

```
from openai import OpenAI, APIError, RateLimitError, AuthenticationError
import time

client = OpenAI()

def safe_completion(prompt: str, retries: int = 3) → str:
    """재시도 로직이 포함된 안전한 API 호출 함수"""
    for attempt in range(retries):
        try:
            response = client.chat.completions.create(
                model="gpt-4o-mini",
                messages=[{"role": "user", "content": prompt}],
                max_tokens=300
            )
            return response.choices[0].message.content or ""

        except AuthenticationError:
            # API 키 오류는 재시도해도 의미 없음
            print("❌ API 키가 유효하지 않습니다. 키를 확인하세요.")
            raise

        except RateLimitError:
            # 요청 한도 초과: 잠시 대기 후 재시도
            wait_time = 2 ** attempt  # 지수 백오프: 1초, 2초, 4초
            print(f"⚠️ 요청 한도 초과. {wait_time}초 후 재시도합니다... ({attempt+1}/{retries})")
            time.sleep(wait_time)

        except APIError as e:
            # 기타 API 오류
            print(f"❌ API 오류 발생: {e}")
            if attempt == retries - 1:
                raise
            time.sleep(1)

    return ""

result = safe_completion("안녕하세요! 오늘 날씨 어때요?")
print(result)
```

> ***주요 예외 클래스 정리*** - *`AuthenticationError` : API 키 인증 실패* - *`RateLimitError` : 분당 요청 수 또는 토큰 한도 초과* - *`APIError` : 서버 측 일반 오류* - *`APIConnectionError` : 네트워크 연결 실패* - *`BadRequestError` : 잘못된 요청 파라미터*

14.5 스트리밍(Streaming) 응답 구현

스트리밍(Streaming)은 모델이 응답을 생성하는 즉시 토큰 단위로 전달받는 방식입니다. 긴 응답을 기다리지 않고 실시간으로 출력할 수 있어 사용자 경험이 크게 향상됩니다.

```
from openai import OpenAI

client = OpenAI()

def stream_completion(prompt: str) → None:
    """스트리밍 방식으로 응답을 출력하는 함수"""
    print("응답: ", end="", flush=True)

    stream = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
        max_tokens=500,
        stream=True  # 스트리밍 활성화
    )

    full_response = ""

    for chunk in stream:
        # delta에 텍스트가 있는 경우만 처리
        delta = chunk.choices[0].delta
        if delta.content:
            print(delta.content, end="", flush=True)
            full_response += delta.content

    print()  # 줄바꿈
    return full_response

stream_completion("대한민국의 수도와 주요 도시 다섯 곳을 알려주세요.")
```

스트리밍 vs 일반 호출 비교

구분	일반 호출	스트리밍 호출
응답 방식	전체 완성 후 한 번에 반환	토큰 단위로 순차 반환
사용자 경험	대기 시간 있음	실시간 출력으로 빠른 체감 속도
적합한 용도	배치 처리, 후처리 필요 시	챗봇 UI, 긴 응답
구현 복잡도	단순	반복문 처리 필요

14.6 대화 이력(Context) 관리 — 멀티턴 대화 구현

멀티턴(multi-turn) 대화를 구현하려면 이전 대화 내용을 메시지 목록에 누적하여 매번 API에 전달해야 합니다. **Chat Completions API**는 상태를 저장하지 않으므로, 컨텍스트 관리는 클라이언트(코드) 쪽에서 직접 수행해야 합니다.

```python
from openai import OpenAI

client = OpenAI()

class ChatSession:
    """대화 이력을 관리하는 멀티턴 챗봇 클래스"""

    def __init__(self, system_prompt: str = "당신은 친절한 AI 어시스턴트입니다."):
        self.messages = [
            {"role": "system", "content": system_prompt}
        ]
        self.model = "gpt-4o-mini"

    def chat(self, user_input: str) → str:
        """사용자 입력을 받아 응답을 반환하고 이력을 갱신합니다."""
        # 사용자 메시지 추가
        self.messages.append({"role": "user", "content": user_input})

        # API 호출
        response = client.chat.completions.create(
            model=self.model,
            messages=self.messages,
            max_tokens=500
        )

        assistant_reply = response.choices[0].message.content or ""

        # 어시스턴트 응답도 이력에 추가
        self.messages.append({"role": "assistant", "content": assistant_reply})

        return assistant_reply

    def reset(self):
        """대화 이력을 초기화합니다 (system 메시지는 유지)."""
        system_msg = self.messages[0]
        self.messages = [system_msg]

    def get_history(self) → list:
        """system 메시지를 제외한 대화 이력을 반환합니다."""
        return [m for m in self.messages if m["role"] ≠ "system"]


# 사용 예시
session = ChatSession(system_prompt="당신은 파이썬 튜터입니다. 간결하게 설명하세요.")
```

```
# 멀티턴 대화
questions = [
    "파이썬 리스트와 튜플의 차이는 무엇인가요?",
    "그럼 언제 튜플을 사용하는 게 좋을까요?",
    "딕셔너리는 어떤 경우에 쓰나요?"
]

for q in questions:
    print(f"\n[사용자]: {q}")
    answer = session.chat(q)
    print(f"[어시스턴트]: {answer}")

print(f"\n총 대화 횟수: {len(session.get_history()) // 2}회")
```

토큰 한도와 컨텍스트 압축

모델마다 처리할 수 있는 **컨텍스트 윈도우(context window)** 한도가 있습니다. 대화가 길어지면 오래된 이력을 제거하거나 요약하는 전략이 필요합니다.

```
def trim_messages(messages: list, max_pairs: int = 10) → list:
    """
    system 메시지는 보존하고, 대화 이력을 최근 N쌍으로 제한합니다.
    """
    system_msgs = [m for m in messages if m["role"] == "system"]
    conversation = [m for m in messages if m["role"] ≠ "system"]

    # 최근 max_pairs * 2개의 메시지만 유지
    if len(conversation) > max_pairs * 2:
        conversation = conversation[-(max_pairs * 2):]

    return system_msgs + conversation
```

14.7 Gemini API 호출 방법

Google Gemini API는 `google-generativeai` 라이브러리를 통해 사용합니다. `gemini-1.5-flash`, `gemini-1.5-pro` 등의 모델을 지원하며, **ChatSession** 방식으로 멀티턴 대화를 관리합니다.

설치

```
pip install google-generativeai
```

기본 호출

```
import google.generativeai as genai

genai.configure(api_key="YOUR_GEMINI_API_KEY")

model = genai.GenerativeModel(
    model_name="gemini-1.5-flash",
    system_instruction="당신은 친절한 AI 어시스턴트입니다."
)

response = model.generate_content("파이썬이란 무엇인가요?")
print(response.text)
```

ChatSession을 이용한 멀티턴 대화

`google-generativeai` 라이브러리는 **ChatSession** 객체를 제공하여 대화 이력을 자동으로 관리합니다.

```python
import google.generativeai as genai

genai.configure(api_key="YOUR_GEMINI_API_KEY")

model = genai.GenerativeModel(
    model_name="gemini-1.5-flash",
    system_instruction="당신은 파이썬 튜터입니다. 간결하게 설명하세요."
)

# ChatSession 시작 (이력 자동 관리)
chat = model.start_chat(history=[])

def gemini_chat(session, user_input: str) → str:
    """Gemini ChatSession으로 대화하는 함수"""
    response = session.send_message(user_input)
    return response.text

# 멀티턴 대화 예시
print(gemini_chat(chat, "리스트 컴프리헨션이 뭔가요?"))
print(gemini_chat(chat, "예제 코드를 보여주세요."))

# 대화 이력 확인
for message in chat.history:
    print(f"[{message.role}]: {message.parts[0].text[:50]}...")
```

주요 파라미터 (GenerationConfig)

```
import google.generativeai as genai

genai.configure(api_key="YOUR_GEMINI_API_KEY")

generation_config = genai.GenerationConfig(
    temperature=0.7,
    max_output_tokens=500,
    top_p=0.95,
    top_k=40  # Gemini 고유 파라미터: 상위 k개 토큰만 고려
)

model = genai.GenerativeModel(
    model_name="gemini-1.5-flash",
    generation_config=generation_config
)

response = model.generate_content("인공지능의 미래를 설명해주세요.")
print(response.text)
```

14.8 Claude API 호출 방법

Anthropic Claude API는 `anthropic` 라이브러리를 통해 사용합니다. **Messages API** 구조를 사용하며, system 메시지가 messages 목록이 아닌 별도 파라미터로 전달되는 점이 OpenAI와 다릅니다.

설치

```
pip install anthropic
```

기본 호출 — Messages API 구조

```
import anthropic

client = anthropic.Anthropic(api_key="YOUR_ANTHROPIC_API_KEY")

response = client.messages.create(
    model="claude-3-5-haiku-20241022",
    max_tokens=500,
    system="당신은 친절한 AI 어시스턴트입니다.",  # system은 별도 파라미터
    messages=[
        {"role": "user", "content": "파이썬이란 무엇인가요?"}
    ]
)

# 응답 텍스트 추출
print(response.content[0].text)
```

OpenAI와의 구조 비교

항목	OpenAI	Anthropic Claude
라이브러리	`openai`	`anthropic`
클라이언트	`OpenAI()`	`Anthropic()`
호출 메서드	`client.chat.completions.create()`	`client.messages.create()`
system 메시지	messages 목록 내 포함	별도 `system` 파라미터
응답 텍스트	`response.choices[0].message.content`	`response.content[0].text`
최대 토큰 파라미터	`max_tokens`	`max_tokens` (필수)

멀티턴 대화 구현

```
import anthropic

client = anthropic.Anthropic()

class ClaudeChatSession:
    """Claude API 멀티턴 대화 세션 클래스"""

    def __init__(self, system_prompt: str = "당신은 친절한 AI 어시스턴트입니다."):
        self.system = system_prompt
        self.messages = []
        self.model = "claude-3-5-haiku-20241022"

    def chat(self, user_input: str) → str:
        """사용자 입력을 받아 응답을 반환합니다."""
        self.messages.append({"role": "user", "content": user_input})

        response = client.messages.create(
            model=self.model,
            max_tokens=500,
            system=self.system,
            messages=self.messages
        )

        assistant_reply = response.content[0].text
        self.messages.append({"role": "assistant", "content": assistant_reply})

        return assistant_reply

# 사용 예시
session = ClaudeChatSession(system_prompt="당신은 파이썬 튜터입니다.")

print(session.chat("파이썬 딕셔너리란 무엇인가요?"))
print(session.chat("딕셔너리에서 키가 없을 때 오류를 피하려면 어떻게 하나요?"))
```

스트리밍 응답

```
import anthropic

client = anthropic.Anthropic()

def claude_stream(prompt: str) → str:
    """Claude 스트리밍 응답 함수"""
    full_response = ""
    print("응답: ", end="", flush=True)

    with client.messages.stream(
        model="claude-3-5-haiku-20241022",
        max_tokens=500,
        messages=[{"role": "user", "content": prompt}]
    ) as stream:
        for text in stream.text_stream:
            print(text, end="", flush=True)
            full_response += text

    print()
    return full_response

claude_stream("대한민국의 주요 관광지를 다섯 곳 추천해주세요.")
```

[실습] 동일 프롬프트로 GPT / Gemini / Claude 응답 비교 프로그램

세 가지 AI API를 동일한 프롬프트로 호출하여 각 모델의 응답을 나란히 비교하는 프로그램을 만들어 봅니다.

사전 준비

```
pip install openai google-generativeai anthropic
```

환경 변수 설정 (`.env` 파일 또는 OS 환경 변수):

```
OPENAI_API_KEY=your_openai_key
GEMINI_API_KEY=your_gemini_key
ANTHROPIC_API_KEY=your_anthropic_key
```

비교 프로그램 전체 코드

```
import os
import time
from openai import OpenAI
import google.generativeai as genai
import anthropic

# ── API 클라이언트 초기화 ──────────────────────────────────────
openai_client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

genai.configure(api_key=os.getenv("GEMINI_API_KEY"))
gemini_model = genai.GenerativeModel(model_name="gemini-1.5-flash")

claude_client = anthropic.Anthropic(api_key=os.getenv("ANTHROPIC_API_KEY"))


# ── 각 API 호출 함수 ──────────────────────────────────────────

def call_gpt(prompt: str, system: str = "") → dict:
    """OpenAI GPT 호출"""
    messages = []
    if system:
        messages.append({"role": "system", "content": system})
    messages.append({"role": "user", "content": prompt})

    start = time.time()
    response = openai_client.chat.completions.create(
        model="gpt-4o-mini",
        messages=messages,
        max_tokens=500,
        temperature=0.7
    )
    elapsed = time.time() - start

    return {
        "model": "GPT-4o mini",
        "response": response.choices[0].message.content,
        "tokens": response.usage.total_tokens,
        "elapsed": round(elapsed, 2)
    }


def call_gemini(prompt: str, system: str = "") → dict:
```

```
    """Google Gemini 호출"""
    model = genai.GenerativeModel(
        model_name="gemini-1.5-flash",
        system_instruction=system if system else None,
        generation_config=genai.GenerationConfig(
            temperature=0.7,
            max_output_tokens=500
        )
    )

    start = time.time()
    response = model.generate_content(prompt)
    elapsed = time.time() - start

    return {
        "model": "Gemini 1.5 Flash",
        "response": response.text,
        "tokens": response.usage_metadata.total_token_count if hasattr(response, "usage_metadata")
else "N/A",
        "elapsed": round(elapsed, 2)
    }


def call_claude(prompt: str, system: str = "") → dict:
    """Anthropic Claude 호출"""
    kwargs = {
        "model": "claude-3-5-haiku-20241022",
        "max_tokens": 500,
        "messages": [{"role": "user", "content": prompt}]
    }
    if system:
        kwargs["system"] = system

    start = time.time()
    response = claude_client.messages.create(**kwargs)
    elapsed = time.time() - start

    return {
        "model": "Claude 3.5 Haiku",
        "response": response.content[0].text,
        "tokens": response.usage.input_tokens + response.usage.output_tokens,
        "elapsed": round(elapsed, 2)
    }
```

```python
# ── 비교 실행 함수 ──────────────────────────────────────────────

def compare_models(prompt: str, system: str = "") → None:
    """세 모델에 동일한 프롬프트를 전송하고 결과를 비교 출력합니다."""
    print("=" * 60)
    print(f"프롬프트: {prompt}")
    if system:
        print(f"시스템:   {system}")
    print("=" * 60)

    results = []
    for call_fn in [call_gpt, call_gemini, call_claude]:
        try:
            result = call_fn(prompt, system)
            results.append(result)
        except Exception as e:
            results.append({"model": call_fn.__name__, "response": f"오류: {e}", "tokens": 0, "elaps
ed": 0})

    for r in results:
        print(f"\n▶ [{r['model']}] (소요: {r['elapsed']}초, 토큰: {r['tokens']})")
        print("-" * 40)
        print(r["response"])

    print("\n" + "=" * 60)
    print("📊 응답 속도 비교")
    print("-" * 40)
    for r in sorted(results, key=lambda x: x["elapsed"]):
        bar = "█" * int(r["elapsed"] * 5)
        print(f"  {r['model']:<20} {r['elapsed']:>5}초  {bar}")
    print("=" * 60)


# ── 메인 실행 ──────────────────────────────────────────────────

if __name__ == "__main__":
    # 테스트 1: 일반 지식 질문
    compare_models(
        prompt="인공지능이 사회에 미치는 긍정적 영향을 세 가지로 요약해주세요.",
        system="당신은 명확하고 간결하게 답변하는 AI 어시스턴트입니다."
    )

    print()

    # 테스트 2: 코드 생성
```

```
    compare_models(
        prompt="파이썬으로 피보나치 수열을 생성하는 함수를 작성해주세요.",
        system="당신은 파이썬 전문가입니다. 코드에 주석을 포함하세요."
    )

    print()

    # 테스트 3: 창의적 질문
    compare_models(
        prompt="만약 인터넷이 갑자기 사라진다면 세상은 어떻게 변할까요?",
        system=""  # system 없이 모델 기본 설정 사용
    )
```

실행 결과 예시

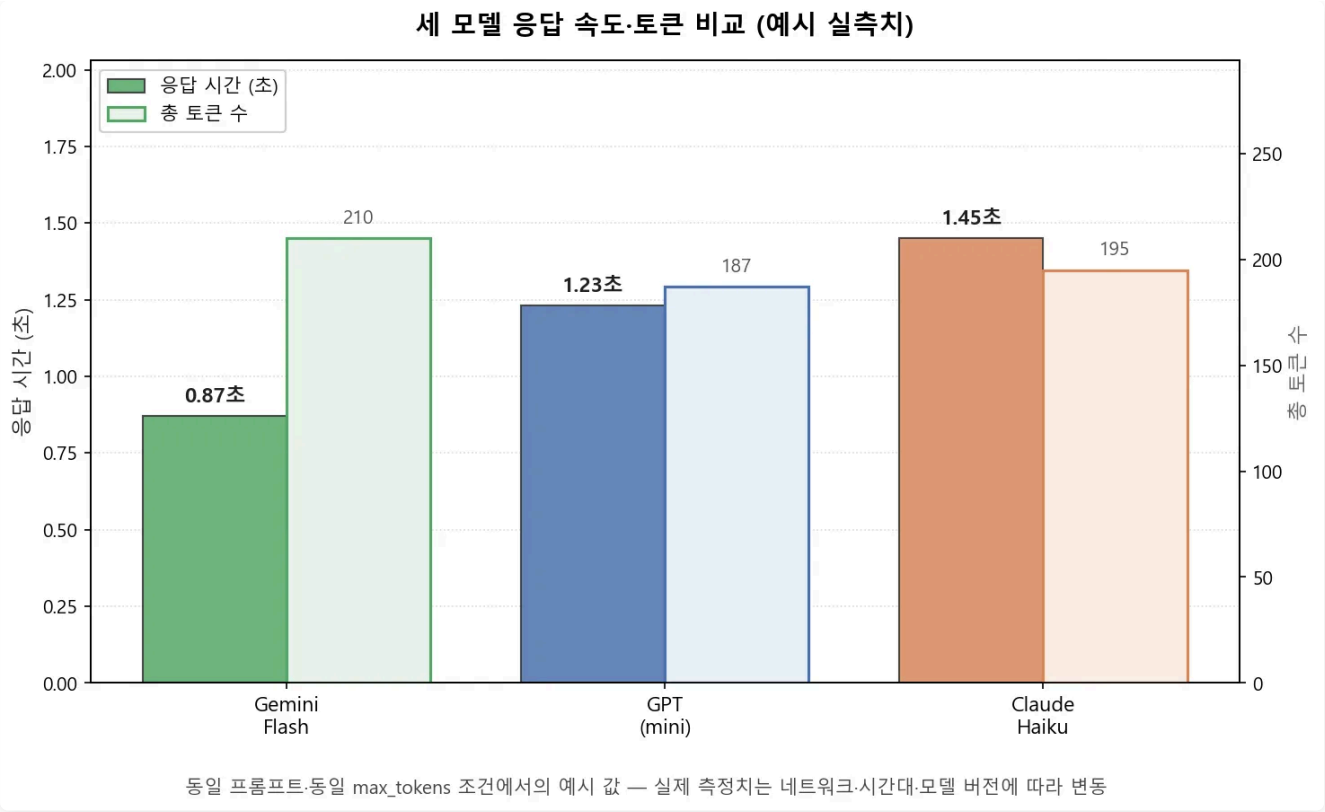
```
============================================================
프롬프트: 인공지능이 사회에 미치는 긍정적 영향을 세 가지로 요약해주세요.
시스템:   당신은 명확하고 간결하게 답변하는 AI 어시스턴트입니다.
============================================================

▶ [GPT-4o mini] (소요: 1.23초, 토큰: 187)
----------------------------------------
1. 의료 진단 향상 ...

▶ [Gemini 1.5 Flash] (소요: 0.87초, 토큰: 210)
----------------------------------------
1. 업무 자동화 ...

▶ [Claude 3.5 Haiku] (소요: 1.45초, 토큰: 195)
----------------------------------------
1. 생산성 향상 ...

============================================================
📊 응답 속도 비교
============================================================
```



> ***실습 확장 아이디어*** *- 응답을 파일이나 데이터베이스에 저장하여 나중에 비교 분석합니다. -* `asyncio` *를 사용해 세 API를 병렬로 호출하면 전체 소요 시간을 줄일 수 있습니다. - temperature 값을 0.0, 0.7, 1.5로 바꾸며 동일 모델의 응답 변화를 관찰합니다.*

정리

이번 챕터에서는 **Chat Completions API**의 구조부터 멀티턴 대화 구현, 스트리밍 응답, 그리고 Gemini와 Claude API 활용까지 폭넓게 살펴보았습니다.

항목	OpenAI GPT	Google Gemini	Anthropic Claude
라이브러리	`openai`	`google-generativeai`	`anthropic`
핵심 메서드	`chat.completions.create()`	`generate_content()`	`messages.create()`
멀티턴 관리	수동 (messages 리스트)	`start_chat()` 자동	수동 (messages 리스트)
system 메시지	messages 내 포함	`system_instruction`	별도 `system` 파라미터
스트리밍	`stream=True`	`stream=True`	`messages.stream()`

세 API 모두 유사한 개념을 사용하지만 세부 구조가 다르므로, 이번 챕터의 비교 프로그램을 직접 실행하며 차이점을 몸으로 익혀두면 이후 프로젝트에서 큰 도움이 됩니다.

다음 챕터에서는 OpenAI의 **Responses API**를 살펴보며 Chat Completions API와의 차이점, 그리고 함수 호출(Function Calling)과 같은 고급 기능을 알아봅니다.

이전 챕터에서는 **Chat Completions API**를 사용하여 대화형 AI 애플리케이션을 만드는 방법을 배웠습니다. 이번 챕터에서는 OpenAI가 새롭게 선보인 **Responses API**를 소개하고, 기존 API와의 차이점, 마이그레이션 방법을 살펴봅니다. 또한 이미지와 파일을 함께 처리하는 **멀티모달(Multimodal)** 기능을 GPT, Gemini, Claude 세 가지 모델로 비교하며 실습합니다.

> *이 챕터를 마치면 최신 Responses API를 자유롭게 다루고, 텍스트와 이미지를 함께 처리하는 멀티모달 챗봇을 직접 만들 수 있습니다.*
>
> 📌 ***[2026 업데이트] 최신 모델 매핑***
>
> *이 챕터의 코드 예시는 Responses API가 공개된 2025년 3월 시점 모델 ID(`gpt-4o`, `gemini-1.5-*`, `claude-opus-4-5`)를 사용합니다.* ***2026년 5월 기준 동급 최신 모델:***
>
챕터 코드 ID	*2026년 동급 최신 모델*
`gpt-4o`	`gpt-5` *(Responses API 권장)*
`gemini-1.5-flash`	`gemini-2.5-flash`
`gemini-1.5-pro`	`gemini-2.5-pro` *또는* `gemini-3-pro`
`claude-opus-4-5`	`claude-opus-4-7` *(1M 컨텍스트)*
>
> *Gemini SDK는 `google.generativeai` → `google-genai` 로 마이그레이션이 권장됩니다. OpenAI는 멀티모달·도구 호출·추론을 Responses API + GPT-5로 단일화하는 추세입니다.*

15.1 OpenAI Responses API 소개와 등장 배경

왜 새로운 API가 필요했을까?

2025년 3월, OpenAI는 기존 **Chat Completions API**를 대체하는 **Responses API**를 공개했습니다. 이 변화는 단순한 인터페이스 개편이 아니라, AI 에이전트(Agent) 시대에 맞는 새로운 패러다임으로의 전환을 의미합니다.

기존 Chat Completions API는 한 번의 요청-응답(Request-Response) 구조로 설계되었습니다. 하지만 실제 서비스에서는 다음과 같은 한계가 드러났습니다.

- **상태 관리 부재**: 매 요청마다 전체 대화 기록을 다시 전송해야 했습니다.

- **도구 연동 복잡성**: 웹 검색, 코드 실행 등 내장 도구를 사용하려면 별도의 복잡한 처리가 필요했습니다.
- **에이전트 흐름 지원 미흡**: 멀티스텝 작업을 처리하는 에이전트 패턴을 구현하기 어려웠습니다.

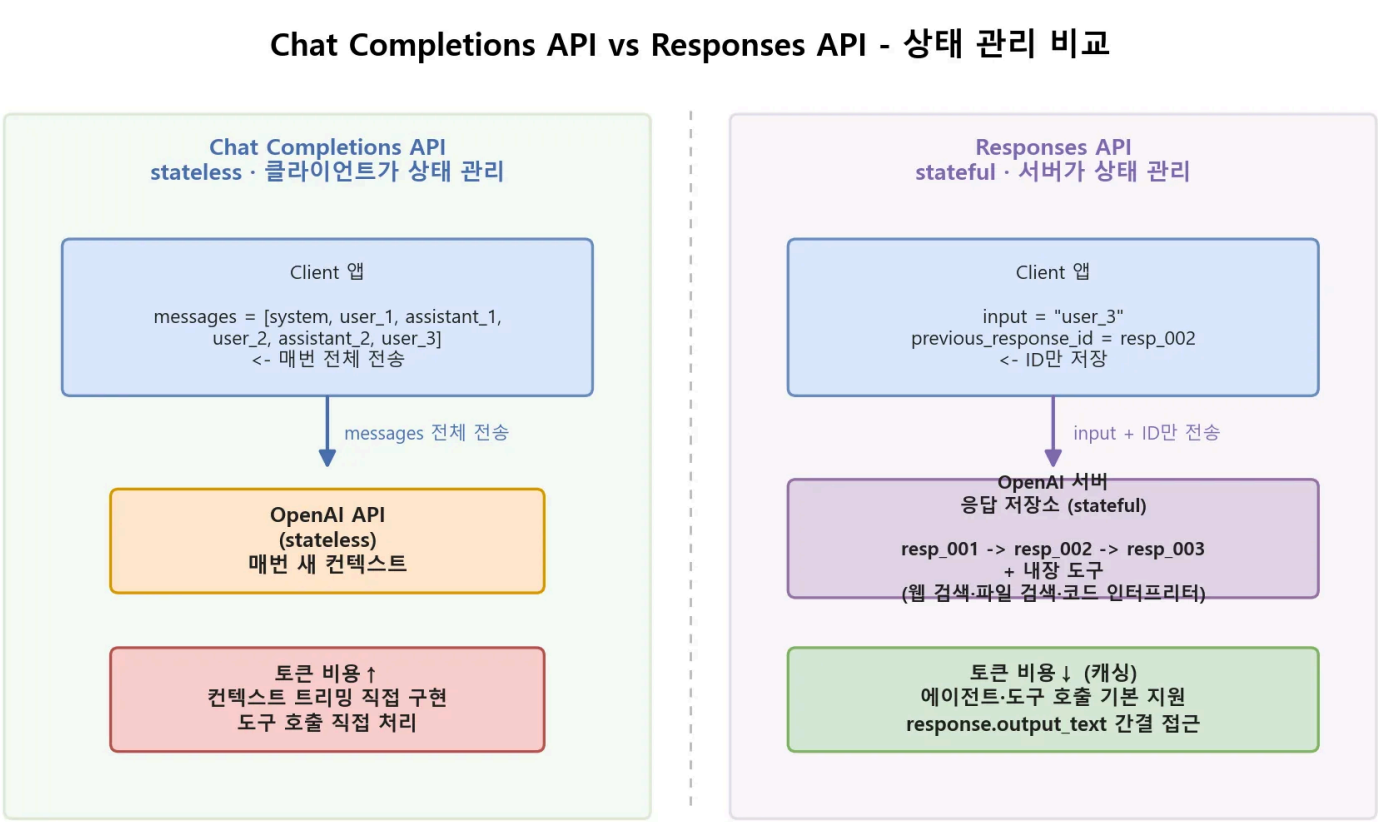
Responses API는 이러한 문제를 해결하기 위해 다음 특징을 중심으로 설계되었습니다.

특징	설명
서버 사이드 상태 관리	대화 기록을 OpenAI 서버가 저장하고 관리합니다.
내장 도구(Built-in Tools)	웹 검색, 파일 검색, 코드 인터프리터를 기본 제공합니다.
에이전트 친화적 구조	멀티스텝 추론과 도구 호출 흐름을 자연스럽게 지원합니다.
통합 인터페이스	Assistants API, Chat Completions API의 기능을 하나로 통합합니다.

> ***참고**: OpenAI는 Responses API가 기존 Chat Completions API를 완전히 대체하는 것은 아니라고 밝혔습니다. 단순한 단일 턴 요청에는 Chat Completions API도 계속 사용할 수 있습니다.*

15.2 Chat Completions API vs Responses API 차이점

두 API의 핵심 차이점을 표로 정리합니다.



구분	Chat Completions API	Responses API
상태 관리	클라이언트(개발자)가 직접 관리	서버(OpenAI)가 자동 관리
대화 기록 전송	매 요청마다 전체 기록 전송	`previous_response_id` 로 연결
내장 도구	없음 (별도 구현 필요)	웹 검색, 파일 검색, 코드 인터프리터 내장
주요 반환 객체	`ChatCompletion`	`Response`
응답 접근 경로	`response.choices[0].message.content`	`response.output_text`
입력 파라미터	`messages`	`input`
모델 파라미터	`model`	`model` (동일)
에이전트 지원	제한적	기본 지원
Assistants API 대체	불가	가능

응답 구조 비교

Chat Completions API 응답 구조:

```
# Chat Completions API
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": "안녕하세요"}]
)

# 응답 접근 방법
text = response.choices[0].message.content
print(text)
```

Responses API 응답 구조:

```
# Responses API
response = client.responses.create(
    model="gpt-4o",
    input="안녕하세요"
)

# 응답 접근 방법 (훨씬 간결)
text = response.output_text
print(text)
```

15.3 마이그레이션 가이드 - 코드 변환 예시

기존 Chat Completions API 코드를 Responses API로 변환하는 방법을 단계별로 살펴보겠습니다.

기본 단일 턴 요청 변환

기존 코드 (Chat Completions API):

```
from openai import OpenAI

client = OpenAI()

response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "system", "content": "당신은 친절한 AI 어시스턴트입니다."},
        {"role": "user", "content": "파이썬의 장점을 3가지 알려주세요."}
    ],
    temperature=0.7,
    max_tokens=500
)

print(response.choices[0].message.content)
```

변환된 코드 (Responses API):

```
from openai import OpenAI

client = OpenAI()

response = client.responses.create(
    model="gpt-4o",
    instructions="당신은 친절한 AI 어시스턴트입니다.",  # system 역할 대체
    input="파이썬의 장점을 3가지 알려주세요.",          # user 메시지
    temperature=0.7,
    max_output_tokens=500                              # max_tokens → max_output_tokens
)

print(response.output_text)
```

다중 턴 대화 변환

기존 코드 (Chat Completions API):

```
from openai import OpenAI

client = OpenAI()

# 대화 기록을 개발자가 직접 관리
conversation_history = [
    {"role": "system", "content": "당신은 친절한 AI 어시스턴트입니다."}
]

def chat(user_message):
    conversation_history.append({"role": "user", "content": user_message})

    response = client.chat.completions.create(
        model="gpt-4o",
        messages=conversation_history  # 매번 전체 기록 전송
    )

    assistant_message = response.choices[0].message.content
    conversation_history.append({"role": "assistant", "content": assistant_message})

    return assistant_message

print(chat("파이썬이란 무엇인가요?"))
print(chat("그렇다면 주요 활용 분야는?"))  # 앞 대화 맥락 유지
```

변환된 코드 (Responses API):

```python
from openai import OpenAI

client = OpenAI()

# 대화 기록을 서버가 관리 - 개발자는 response_id만 저장
previous_response_id = None

def chat(user_message):
    global previous_response_id

    kwargs = {
        "model": "gpt-4o",
        "instructions": "당신은 친절한 AI 어시스턴트입니다.",
        "input": user_message,
    }

    # 이전 응답이 있으면 연결
    if previous_response_id:
        kwargs["previous_response_id"] = previous_response_id

    response = client.responses.create(**kwargs)

    # 다음 요청을 위해 현재 응답 ID 저장
    previous_response_id = response.id

    return response.output_text

print(chat("파이썬이란 무엇인가요?"))
print(chat("그렇다면 주요 활용 분야는?"))  # 서버가 자동으로 이전 맥락 참조
```

스트리밍 응답 변환

기존 코드 (Chat Completions API):

```
from openai import OpenAI

client = OpenAI()

stream = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": "AI의 미래를 설명해주세요."}],
    stream=True
)

for chunk in stream:
    if chunk.choices[0].delta.content is not None:
        print(chunk.choices[0].delta.content, end="", flush=True)
```

변환된 코드 (Responses API):

```
from openai import OpenAI

client = OpenAI()

with client.responses.stream(
    model="gpt-4o",
    input="AI의 미래를 설명해주세요."
) as stream:
    for text in stream.text_stream:
        print(text, end="", flush=True)
```

주요 파라미터 변환 요약

Chat Completions API	Responses API	비고
`messages`	`input`	단순 문자열 또는 메시지 배열
`messages[system]`	`instructions`	시스템 프롬프트 전용 파라미터
`max_tokens`	`max_output_tokens`	파라미터명 변경
`response.choices[0].message.content`	`response.output_text`	응답 접근 경로 간소화
없음	`previous_response_id`	서버 사이드 대화 연결
없음	`store=True/False`	응답 저장 여부 제어

15.4 최신 SDK 활용법

SDK 설치 및 버전 확인

Responses API를 사용하려면 **openai Python SDK** 버전 1.66.0 이상이 필요합니다.

```
# SDK 설치 또는 업그레이드
# pip install --upgrade openai

import openai
print(openai.__version__)  # 1.66.0 이상인지 확인
```

내장 도구(Built-in Tools) 활용

Responses API의 가장 강력한 기능 중 하나는 별도 구현 없이 사용할 수 있는 **내장 도구**입니다.

웹 검색 도구 사용 예시:

```
from openai import OpenAI

client = OpenAI()

response = client.responses.create(
    model="gpt-4o",
    tools=[{"type": "web_search_preview"}],  # 웹 검색 도구 활성화
    input="2025년 최신 AI 모델 트렌드를 알려주세요."
)

print(response.output_text)
```

파일 검색 도구 사용 예시:

```
from openai import OpenAI

client = OpenAI()

# 파일 업로드 (사전에 진행)
# file = client.files.create(file=open("document.pdf", "rb"), purpose="assistants")

response = client.responses.create(
    model="gpt-4o",
    tools=[{
        "type": "file_search",
        "vector_store_ids": ["vs_xxxxx"]  # 벡터 스토어 ID
    }],
    input="문서에서 환불 정책을 찾아주세요."
)

print(response.output_text)
```

응답 저장 및 비용 관리

```
from openai import OpenAI

client = OpenAI()

# store=False: 응답을 서버에 저장하지 않음 (개인정보 보호, 비용 절감)
response = client.responses.create(
    model="gpt-4o",
    input="임시 분석 요청입니다.",
    store=False  # 기본값은 True
)

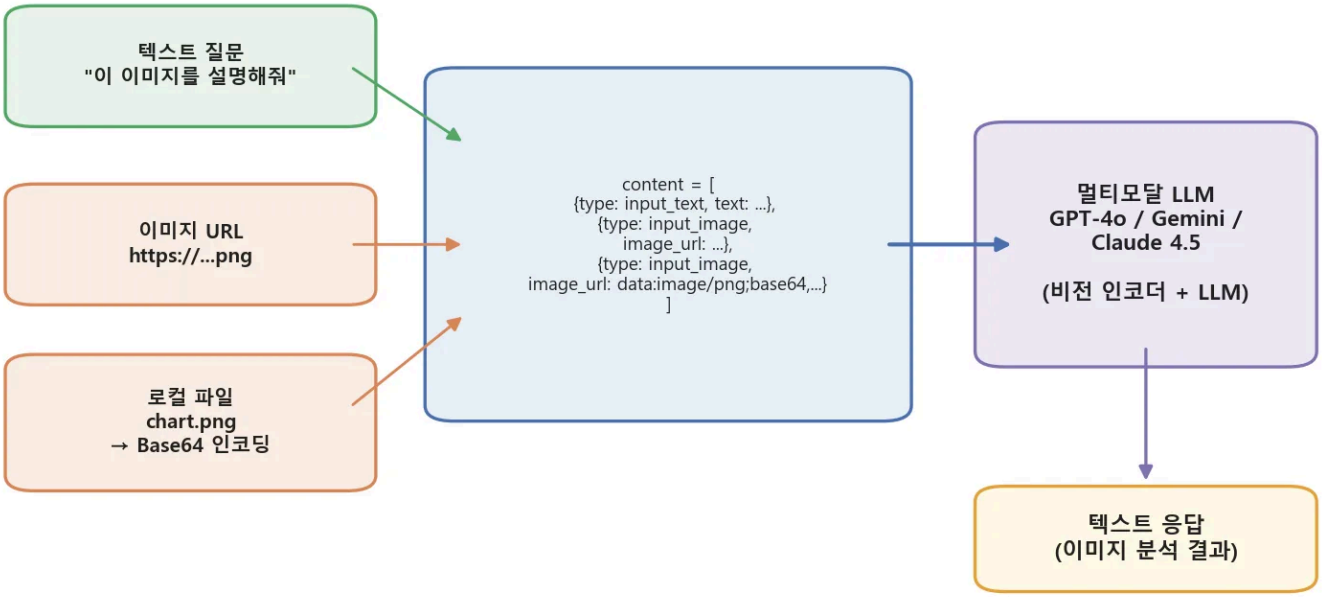
# 저장된 응답 조회
retrieved = client.responses.retrieve(response.id)

# 저장된 응답 삭제
client.responses.delete(response.id)
```

15.5 멀티모달 입력 처리 (이미지, 파일 첨부)

멀티모달(Multimodal)이란 텍스트뿐만 아니라 이미지, 오디오, 파일 등 다양한 형식의 입력을 함께 처리하는 능력을 말합니다. GPT-4o는 이미지를 포함한 멀티모달 입력을 기본으로 지원합니다.

멀티모달 입력 처리 — 텍스트 + 이미지를 함께 LLM에 전달



GPT는 input_image, Gemini는 PIL.Image, Claude는 source.type=base64|url 형식 — 개념은 동일

이미지 URL로 분석하기

```
from openai import OpenAI

client = OpenAI()

response = client.responses.create(
    model="gpt-4o",
    input=[
        {
            "role": "user",
            "content": [
                {
                    "type": "input_text",
                    "text": "이 이미지에서 무엇이 보이나요? 자세히 설명해주세요."
                },
                {
                    "type": "input_image",
                    "image_url": "https://upload.wikimedia.org/wikipedia/commons/thumb/4/47/PNG_transparency_demonstration_1.png/280px-PNG_transparency_demonstration_1.png"
                }
            ]
        }
    ]
)

print(response.output_text)
```

로컬 이미지 파일 분석하기

```
import base64
from openai import OpenAI

client = OpenAI()

def encode_image(image_path: str) → str:
    """이미지 파일을 Base64로 인코딩합니다."""
    with open(image_path, "rb") as image_file:
        return base64.b64encode(image_file.read()).decode("utf-8")

# 로컬 이미지 파일 경로
image_path = "sample_chart.png"
base64_image = encode_image(image_path)

response = client.responses.create(
    model="gpt-4o",
    input=[
        {
            "role": "user",
            "content": [
                {
                    "type": "input_text",
                    "text": "이 차트를 분석하고 주요 인사이트를 알려주세요."
                },
                {
                    "type": "input_image",
                    "image_url": f"{base64_image}"
                }
            ]
        }
    ]
)

print(response.output_text)
```

여러 이미지 동시 분석하기

```
import base64
from openai import OpenAI

client = OpenAI()

def encode_image(image_path: str) → str:
    with open(image_path, "rb") as image_file:
        return base64.b64encode(image_file.read()).decode("utf-8")

# 여러 이미지 분석
image_paths = ["before.png", "after.png"]

content = [
    {
        "type": "input_text",
        "text": "두 이미지의 차이점을 비교하여 설명해주세요."
    }
]

for path in image_paths:
    content.append({
        "type": "input_image",
        "image_url": f"{encode_image(path)}"
    })

response = client.responses.create(
    model="gpt-4o",
    input=[{"role": "user", "content": content}]
)

print(response.output_text)
```

이미지 해상도 제어 (detail 파라미터)

```
from openai import OpenAI

client = OpenAI()

response = client.responses.create(
    model="gpt-4o",
    input=[
        {
            "role": "user",
            "content": [
                {
                    "type": "input_text",
                    "text": "이 이미지의 텍스트를 모두 읽어주세요."
                },
                {
                    "type": "input_image",
                    "image_url": "https://example.com/document.png",
                    "detail": "high"  # "low", "high", "auto" 중 선택
                }
            ]
        }
    ]
)

# detail 파라미터:
# - "auto": 이미지 크기에 따라 자동 선택 (기본값)
# - "low": 빠른 처리, 낮은 비용 (512x512 크롭 사용)
# - "high": 정밀 분석, 높은 비용 (고해상도 처리)

print(response.output_text)
```

15.6 Gemini의 멀티모달 처리 방법

Google의 **Gemini** 모델도 강력한 멀티모달 처리 능력을 갖추고 있습니다. `google-generativeai` 라이브러리를 사용합니다.

Gemini SDK 설치

```
# pip install google-generativeai pillow
```

이미지 URL 분석

```
import google.generativeai as genai
from PIL import Image
import requests
from io import BytesIO

# API 키 설정
genai.configure(api_key="YOUR_GEMINI_API_KEY")

# 모델 초기화
model = genai.GenerativeModel("gemini-1.5-flash")

# URL에서 이미지 로드
image_url = "https://upload.wikimedia.org/wikipedia/commons/thumb/4/47/PNG_transparency_demonstrati
on_1.png/280px-PNG_transparency_demonstration_1.png"
response_img = requests.get(image_url)
image = Image.open(BytesIO(response_img.content))

# 이미지와 텍스트를 함께 전송
response = model.generate_content([
    "이 이미지에서 무엇이 보이나요? 자세히 설명해주세요.",
    image
])

print(response.text)
```

로컬 이미지 파일 분석

```
import google.generativeai as genai
from PIL import Image

genai.configure(api_key="YOUR_GEMINI_API_KEY")

model = genai.GenerativeModel("gemini-1.5-pro")

# 로컬 이미지 파일 열기
image = Image.open("sample_chart.png")

response = model.generate_content([
    "이 차트를 분석하고 주요 인사이트 3가지를 알려주세요.",
    image
])

print(response.text)
```

Gemini로 여러 이미지 비교

```
import google.generativeai as genai
from PIL import Image

genai.configure(api_key="YOUR_GEMINI_API_KEY")

model = genai.GenerativeModel("gemini-1.5-pro")

# 여러 이미지 로드
image1 = Image.open("product_front.jpg")
image2 = Image.open("product_back.jpg")

response = model.generate_content([
    "다음 두 이미지를 비교하여 공통점과 차이점을 분석해주세요.",
    image1,
    image2
])

print(response.text)
```

Gemini 멀티모달 스트리밍

```
import google.generativeai as genai
from PIL import Image

genai.configure(api_key="YOUR_GEMINI_API_KEY")

model = genai.GenerativeModel("gemini-1.5-flash")
image = Image.open("complex_diagram.png")

# 스트리밍으로 응답 받기
for chunk in model.generate_content(
    ["이 다이어그램의 구조를 상세히 설명해주세요.", image],
    stream=True
):
    print(chunk.text, end="", flush=True)
```

15.7 Claude의 Vision 기능 활용

Anthropic의 **Claude** 모델도 강력한 비전(Vision) 기능을 제공합니다. `anthropic` 라이브러리를 사용합니다.

Anthropic SDK 설치

```
# pip install anthropic
```

이미지 URL 분석

```
import anthropic

client = anthropic.Anthropic(api_key="YOUR_ANTHROPIC_API_KEY")

message = client.messages.create(
    model="claude-opus-4-5",
    max_tokens=1024,
    messages=[
        {
            "role": "user",
            "content": [
                {
                    "type": "image",
                    "source": {
                        "type": "url",
                        "url": "https://upload.wikimedia.org/wikipedia/commons/thumb/4/47/PNG_trans
parency_demonstration_1.png/280px-PNG_transparency_demonstration_1.png"
                    }
                },
                {
                    "type": "text",
                    "text": "이 이미지에서 무엇이 보이나요? 자세히 설명해주세요."
                }
            ]
        }
    ]
)

print(message.content[0].text)
```

로컬 이미지 파일 분석 (Base64 인코딩)

```
import anthropic
import base64

client = anthropic.Anthropic(api_key="YOUR_ANTHROPIC_API_KEY")

def encode_image(image_path: str) → tuple[str, str]:
    """이미지를 Base64로 인코딩하고 미디어 타입을 반환합니다."""
    extension = image_path.split(".")[-1].lower()
    media_type_map = {
        "jpg": "image/jpeg",
        "jpeg": "image/jpeg",
        "png": "image/png",
        "gif": "image/gif",
        "webp": "image/webp"
    }
    media_type = media_type_map.get(extension, "image/jpeg")

    with open(image_path, "rb") as f:
        image_data = base64.b64encode(f.read()).decode("utf-8")

    return image_data, media_type

image_data, media_type = encode_image("sample_chart.png")

message = client.messages.create(
    model="claude-opus-4-5",
    max_tokens=1024,
    messages=[
        {
            "role": "user",
            "content": [
                {
                    "type": "image",
                    "source": {
                        "type": "base64",
                        "media_type": media_type,
                        "data": image_data
                    }
                },
                {
                    "type": "text",
                    "text": "이 차트를 분석하고 주요 인사이트를 알려주세요."
```

```
                }
            ]
        }
    ]
)

print(message.content[0].text)
```

세 가지 모델 멀티모달 비교

```
import anthropic
import google.generativeai as genai
from openai import OpenAI
import base64
from PIL import Image
import requests
from io import BytesIO

def analyze_image_gpt(image_url: str, question: str) → str:
    """GPT-4o로 이미지 분석"""
    client = OpenAI()
    response = client.responses.create(
        model="gpt-4o",
        input=[{
            "role": "user",
            "content": [
                {"type": "input_text", "text": question},
                {"type": "input_image", "image_url": image_url}
            ]
        }]
    )
    return response.output_text

def analyze_image_gemini(image_url: str, question: str) → str:
    """Gemini로 이미지 분석"""
    genai.configure(api_key="YOUR_GEMINI_API_KEY")
    model = genai.GenerativeModel("gemini-1.5-flash")

    img_response = requests.get(image_url)
    image = Image.open(BytesIO(img_response.content))

    response = model.generate_content([question, image])
    return response.text

def analyze_image_claude(image_url: str, question: str) → str:
    """Claude로 이미지 분석"""
    client = anthropic.Anthropic(api_key="YOUR_ANTHROPIC_API_KEY")

    message = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=1024,
        messages=[{
```

```
            "role": "user",
            "content": [
                {
                    "type": "image",
                    "source": {"type": "url", "url": image_url}
                },
                {"type": "text", "text": question}
            ]
        }]
    )
    return message.content[0].text

# 비교 실행
IMAGE_URL = "https://upload.wikimedia.org/wikipedia/commons/thumb/4/47/PNG_transparency_demonstrati
on_1.png/280px-PNG_transparency_demonstration_1.png"
QUESTION = "이 이미지를 설명해주세요."

print("=== GPT-4o ===")
print(analyze_image_gpt(IMAGE_URL, QUESTION))

print("\n=== Gemini 1.5 Flash ===")
print(analyze_image_gemini(IMAGE_URL, QUESTION))

print("\n=== Claude Opus ===")
print(analyze_image_claude(IMAGE_URL, QUESTION))
```

[실습] Responses API로 멀티모달 챗봇 만들기

이번 실습에서는 **Responses API**의 서버 사이드 상태 관리와 **멀티모달** 처리 기능을 결합하여 이미지를 이해하고 대화를 이어가는 챗봇을 만들어 보겠습니다.

실습 목표

- Responses API의 `previous_response_id` 를 활용한 대화 연속성 구현
- 이미지와 텍스트를 함께 처리하는 멀티모달 챗봇 구현
- 간단한 CLI 인터페이스로 인터랙티브 챗봇 완성

전체 코드

```
import base64
import os
from openai import OpenAI

client = OpenAI(api_key=os.environ.get("OPENAI_API_KEY"))

def encode_image_to_base64(image_path: str) → str:
    """로컬 이미지 파일을 Base64 문자열로 인코딩합니다."""
    with open(image_path, "rb") as image_file:
        return base64.b64encode(image_file.read()).decode("utf-8")

def build_user_message(text: str, image_source: str | None = None) → list:
    """
    사용자 메시지 콘텐츠 블록을 생성합니다.
    image_source가 있으면 이미지를 포함합니다.
    """
    content = []

    if image_source:
        # URL인지 로컬 파일인지 판단
        if image_source.startswith("http://") or image_source.startswith("https://"):
            content.append({
                "type": "input_image",
                "image_url": image_source
            })
        else:
            # 로컬 파일 → Base64 인코딩
            base64_image = encode_image_to_base64(image_source)
            # 확장자로 MIME 타입 결정
            ext = image_source.split(".")[-1].lower()
            mime_map = {"jpg": "jpeg", "jpeg": "jpeg", "png": "png", "gif": "gif", "webp": "webp"}
            mime = mime_map.get(ext, "jpeg")
            content.append({
                "type": "input_image",
                "image_url": f"{base64_image}"
            })

    content.append({
        "type": "input_text",
        "text": text
    })
```

```
    return content

def multimodal_chatbot():
    """
    Responses API를 사용한 멀티모달 챗봇 메인 함수.
    서버 사이드 상태 관리로 대화 기록을 자동 유지합니다.
    """
    print("=" * 60)
    print("  멀티모달 챗봇 (Responses API)")
    print("=" * 60)
    print("명령어 안내:")
    print("  이미지 분석: '이미지: [경로 또는 URL]' 입력 후 Enter")
    print("  일반 대화:   텍스트를 입력하고 Enter")
    print("  종료:        'quit' 또는 'exit' 입력")
    print("-" * 60)

    previous_response_id = None  # 대화 연결을 위한 이전 응답 ID

    while True:
        user_input = input("\n나: ").strip()

        if not user_input:
            continue

        if user_input.lower() in ["quit", "exit", "종료"]:
            print("챗봇을 종료합니다. 안녕히 가세요!")
            break

        # 이미지 포함 여부 확인
        image_source = None
        message_text = user_input

        if user_input.lower().startswith("이미지:"):
            parts = user_input.split(" ", 1)
            if len(parts) > 1:
                image_source = parts[1].strip()
                message_text = input("이미지에 대해 질문하세요: ").strip()
                if not message_text:
                    message_text = "이 이미지를 자세히 설명해주세요."

        # 메시지 콘텐츠 블록 생성
        content = build_user_message(message_text, image_source)

        # Responses API 요청 구성
        request_params = {
```

```
            "model": "gpt-4o",
            "instructions": (
                "당신은 도움이 되는 AI 어시스턴트입니다. "
                "이미지가 제공되면 상세히 분석하고, "
                "이전 대화 맥락을 유지하며 자연스럽게 대화합니다. "
                "한국어로 답변합니다."
            ),
            "input": [{"role": "user", "content": content}]
        }

        # 이전 응답이 있으면 대화 연결
        if previous_response_id:
            request_params["previous_response_id"] = previous_response_id

        try:
            print("\nAI: ", end="", flush=True)

            # 스트리밍으로 응답 받기
            with client.responses.stream(**request_params) as stream:
                response_text = ""
                for text in stream.text_stream:
                    print(text, end="", flush=True)
                    response_text += text

                # 완료된 응답에서 ID 추출
                final_response = stream.get_final_response()
                previous_response_id = final_response.id

            print()  # 줄바꿈

        except Exception as e:
            print(f"\n오류가 발생했습니다: {e}")
            print("다시 시도해주세요.")

if __name__ == "__main__":
    multimodal_chatbot()
```

실행 방법

```
# 환경 변수 설정
export OPENAI_API_KEY="your-api-key-here"

# 챗봇 실행
python multimodal_chatbot.py
```

실행 예시

```
============================================================
  멀티모달 챗봇 (Responses API)
============================================================
명령어 안내:
  이미지 분석: '이미지: [경로 또는 URL]' 입력 후 Enter
  일반 대화:   텍스트를 입력하고 Enter
  종료:        'quit' 또는 'exit' 입력
------------------------------------------------------------

나: 이미지: https://example.com/chart.png
이미지에 대해 질문하세요: 이 차트의 전체적인 트렌드를 알려주세요.

AI: 이 차트는 2020년부터 2024년까지의 매출 추이를 보여주고 있습니다.
전반적으로 상승 트렌드를 보이며, 특히 2022년에 급격한 성장이 있었습니다...

나: 가장 성장률이 높은 분기는 언제인가요?

AI: 앞서 보여드린 차트를 바탕으로 분석하면, 2022년 3분기의 성장률이
가장 높았습니다. 이 기간에는 전 분기 대비 약 35% 성장을 기록했습니다...
```

핵심 포인트 정리

이번 실습에서 구현한 핵심 기능은 다음과 같습니다.

기능	구현 방법
대화 연속성	`previous_response_id` 로 서버 사이드 대화 기록 연결
이미지 URL 처리	`input_image` 타입에 URL 직접 전달
로컬 파일 처리	Base64 인코딩 후 Data URL 형식으로 전달
실시간 출력	`client.responses.stream()` 컨텍스트 매니저 활용
오류 처리	`try-except` 로 API 오류 안전하게 처리

정리

이번 챕터에서는 OpenAI의 최신 **Responses API**와 멀티모달 처리 기능을 학습했습니다.

- **Responses API**는 서버 사이드 상태 관리, 내장 도구, 에이전트 지원을 통해 더 강력하고 간결한 AI 애플리케이션 개발을 가능하게 합니다.
- **Chat Completions API**에서 Responses API로의 마이그레이션은 주요 파라미터명 변경과 응답 접근 방식 수정으로 비교적 간단하게 진행할 수 있습니다.
- **GPT-4o**, **Gemini**, **Claude** 모두 멀티모달 처리를 지원하며, 각각 고유한 API 인터페이스를 갖습니다.
- 실습에서는 Responses API의 `previous_response_id` 를 활용해 대화 맥락을 유지하는 멀티모달 챗봇을 구현했습니다.

다음 챕터에서는 **Function Calling**을 학습합니다. AI 모델이 외부 함수와 API를 호출하는 방법을 통해 더욱 실용적인 애플리케이션을 만들어 보겠습니다.